Threat Model — Logfire Python SDK

Date: 2025-07-14 **Version**: 1.0 **SDK Version**: 4.31.0 **Scope**: Open-source Python SDK (`logfire` package) — telemetry collection, credential management, CLI, auto-instrumentation, and data export to the Logfire SaaS backend. The closed-source Logfire platform (server, dashboard, storage) is **out of scope**. **Methodology**: Microsoft SDL Design-phase threat modeling — STRIDE per component, DREAD scoring, MSPP severity classification. **Author**: Threat Modeling Agent (SDL-aligned)

Table of Contents

1. Scope & Objectives
2. Assets
3. Data Flow Diagram
4. Trust Boundaries
5. Entry Points
6. STRIDE Analysis Per Component
7. Threat Register
8. Mitigations Summary
9. Out of Scope

---

1. Scope & Objectives

System Under Review

The **Logfire Python SDK** (`logfire` v4.31.0) is an OpenTelemetry distribution and observability library. It is distributed on PyPI and installed into user applications. At runtime it:

- Collects traces, structured logs, and metrics from the host Python application.
- Auto-instruments third-party frameworks (FastAPI, Django, SQLAlchemy, OpenAI, Anthropic, MCP, and ~30 others).
- Serialises telemetry as OTLP/Protobuf and exports it to the Logfire SaaS backend over HTTPS.
- Manages authentication credentials (write tokens, read tokens, user tokens, API keys) locally on disk and via environment variables.
- Provides a CLI (`logfire`) for authentication, project management, and running scripts under auto-instrumentation.

Security Objectives

#	Objective	Property
SO-1	Write tokens and API keys must not leak outside the authorised application process.	Confidentiality
SO-2	Telemetry data sent to the backend must arrive intact and unmodified.	Integrity
SO-3	Sensitive application data (PII, credentials, query parameters) must be redacted before export.	Confidentiality
SO-4	Telemetry must reach the intended and authorised backend only.	Integrity / Authentication
SO-5	The SDK's auto-instrumentation must not introduce remote code execution or privilege escalation vectors.	Authorization

#	Objective	Property
SO-6	The SDK's package supply chain must be tamper-resistant.	Integrity
SO-7	CLI authentication credentials must be stored securely on disk.	Confidentiality

Assumptions

- The host OS is trusted for process isolation. Local privilege escalation to the same UID is treated as a separate threat plane.
- TLS certificate validation is enabled by default in the underlying `requests` library unless explicitly overridden by the application.
- The Logfire SaaS backend enforces its own server-side authorisation; the SDK cannot compensate for backend vulnerabilities.
- Users who call `logfire.configure(scrubbing=False)` or `logfire.configure(token=...)` in source code do so knowingly.

---

2. Assets

ID	Asset	Description	Sensitivity
A-01	**LOGFIRE_TOKEN / write tokens**	Static bearer tokens embedded in every OTLP export `Authorization` header. Grant unlimited telemetry ingest into the associated project. Format: `pylf_v{version}_{region}_{random}`.	**HBI**
A-02	**LOGFIRE_API_KEY**	Credential for the managed-variables (remote config) API.	**HBI**

ID	Asset	Description	Sensitivity
A-03	**User tokens** (`~/.logfire/default.toml`)	OAuth device-flow tokens authorising management operations (project creation, token issuance). Stored in plaintext TOML.	**HBI**
A-04	**Read tokens**	Issued via `logfire read-tokens create`. Authorise SQL queries against all telemetry in a project.	**HBI**
A-05	**Project credentials** (`.logfire/logfire_credentials.json`)	Per-project JSON file containing write token and backend URL.	**HBI**
A-06	**Telemetry payload (in transit / on disk)**	OTel spans, logs, and metrics containing application behaviour, potentially including user data, query parameters, HTTP request/response bodies, and LLM prompts. Post-scrubbing but may still contain MBI data.	**MBI–HBI**
A-07	**Disk retry queue** (`/tmp/logfire-retryer-*/`)	Failed OTLP export payloads (raw Protobuf bytes) persisted in OS temp dir. Post-scrubbing but spans may contain sensitive attributes. `Authorization` header (write token) is stored in the DiskRetryer session headers object in memory; the raw POST kwargs are stored per task (may include the `Authorization` header value).	**HBI**
A-08	**LLM prompt / response content**	When AI instrumentation is enabled, full prompt text and model responses are captured as span attributes. May contain PII, business secrets, or user-supplied content.	**HBI**
A-09	**SQL query parameters**	DB instrumentation (SQLAlchemy, psycopg, asyncpg, etc.) can capture full SQL statements including parameter values, which may include PII or confidential data.	**MBI–HBI**

ID	Asset	Description	Sensitivity
A-10	**Auto-traced function arguments**	When `install_auto_tracing()` is active, all function arguments and return values in matched modules are captured as span attributes.	**MBI–HBI**
A-11	**CLI activity log** (`~/.logfire/log.txt`)	Python `logging.FileHandler` output during CLI execution. May include request/response metadata.	**LBI–MBI**
A-12	**Package integrity / supply chain**	The `logfire` and `logfire-api` PyPI packages; `uv.lock` lockfile; 30+ OTel instrumentation dependencies.	**HBI** (code execution)

---

3. Data Flow Diagram

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  DEVELOPER WORKSTATION / APPLICATION SERVER                               ║
║                                                                           ║
║  [Developer / CI]                                                        ║
║       │                                                                  ║
║       │ LOGFIRE_TOKEN, LOGFIRE_API_KEY env vars                          ║
║       │ ~/.logfire/default.toml  (user tokens)                          ║
║       │ .logfire/logfire_credentials.json                                ║
║       ▼                                                                  ║
║  ┌──────────────────────────────────────────────────────────────────┐    ║
║  │  (logfire.configure())  ← ConfigParam / ParamManager             │    ║
║  │   • reads env vars, pyproject.toml [tool.logfire], cred files    │    ║
║  │   • validates token format (PYDANTIC_LOGFIRE_TOKEN_PATTERN)      │    ║
║  │   • resolves LOGFIRE_BASE_URL from token region                  │    ║
║  └──────────────┬──────────────────────────┬──────────────────────┘    ║
║                │                          │                              ║
```

```
│              ▼                          ▼                                  │
│  ┌──────────────────────┐   ┌──────────────────────────────────────────┐ │
│  │ (ProxyTracerProvider│   │ (Auto-trace Import Hook)                 │ │
│  │  / ProxyMeterProvider   │  sys.meta_path LogfireFinder +           │ │
│  │  / ProxyLogProvider)│   │  AST rewriter — wraps every fn with      │ │
│  └──────────────────────┘   │  logfire.span()                          │ │
│             │               └──────────────────────────────────────────┘ │
│             │                              │                              │
│    ┌────────▼──────────────────────────── ▼──────────────────────┐      │
│    │  (Host Application)                                          │      │
│    │                                                              │      │
│    │  logfire.span() / .info() / .error() / @instrument           │      │
│    │  instrument_fastapi / instrument_openai / instrument_sqlalchemy│      │
│    │  LogfireLoggingHandler / StructlogProcessor / LogfireHandler  │      │
│    │  Pydantic plugin (implicit on pydantic import)               │      │
│    └──────────────────────────────────────────────────────────────┘      │
│                           │                                                │
│                           │ spans / logs / metrics                         │
│                           ▼                                                │
│              ┌──────────────────────────────┐                             │
│              │ =In-memory OTel Buffers=     │                             │
│              │  BatchSpanProcessor          │                             │
│              │  BatchLogRecordProcessor     │                             │
│              └──────────────────────────────┘                             │
│                           │                                                │
│                           │ export (OTLP/Protobuf)                         │
│                           ▼                                                │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │  (BodySizeCheckingOTLPSpanExporter / OTLPExporterHttpSession)    │    │
│  │   Scrubbing processor runs before export                          │    │
│  │   Authorization: <LOGFIRE_TOKEN> header on every POST             │    │
│  │                                                                  |    │
│  │   On failure ──► =DiskRetryer queue=                              │    │
│  │                  /tmp/logfire-retryer-*/                          │    │
│  │                  raw Protobuf bytes (no encryption)               │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                           │                                                │
```

```
╚═══════════════════════════════╪═══════════════════════════════════════════╝
                                │
        TRUST BOUNDARY ─────────┤ (Application ↔ Internet / Logfire Platform)
                                │ HTTPS / OTLP/HTTP POST
                                ▼
           ┌──────────────────────────────────────┐
           │  [Logfire Platform]                  │
           │   logfire-us.pydantic.dev (GCP us-east4) │
           │   logfire-eu.pydantic.dev (GCP eu-west4) │
           │   /v1/traces, /v1/logs, /v1/metrics   │
           └──────────────────────────────────────┘

        TRUST BOUNDARY ─────────┐
                                │ HTTPS (management / query / variables)
        ┌───────────────────────┼──────────────────────────────────────┐
        │                       ▼                                      │
        │  [Logfire Management API]    [Logfire Query API]             │
        │   /v1/device-auth/new/        /v1/query (SQL over HTTP)      │
        │   /v1/device-auth/wait/...    Authorization: <read_token>    │
        │   /v1/projects/...                                           │
        │   Authorization: <user_token> [Logfire Variables API]        │
        │                               long-poll background thread    │
        └──────────────────────────────────────────────────────────────┘

                                ┌──────────────────────────────────────┐
   TRUST BOUNDARY ──────────────┤  [PyPI / Package Registry]           │
   (Supply Chain)               │   logfire, logfire-api,              │
                                │   opentelemetry-sdk, opentelemetry-  │
                                │   exporter-otlp-proto-http, rich,    │
                                │   protobuf, executing, 30+ OTel      │
                                │   instrumentation packages           │
                                └──────────────────────────────────────┘

                                ┌──────────────────────────────────────┐
   TRUST BOUNDARY ──────────────┤  [External AI / DB Services]         │
   (AI Instrumentation)         │   OpenAI, Anthropic, Google Gemini,  │
```

```
                              │   LiteLLM, DSPy, PydanticAI, MCP     │
                              │   (LLM prompts/responses captured     │
                              │    into spans by instrumentation)     │
                              └──────────────────────────────────────┘


  TRUST BOUNDARY ──────────────┐
  (User Input ↔ CLI)           ▼
                     [Developer / User]
                      logfire auth / run / projects
                      LOGFIRE_TOKEN, LOGFIRE_BASE_URL env vars
                      pyproject.toml [tool.logfire]
```

---

4. Trust Boundaries

ID	Boundary	Crosses
TB-1	**Application ↔ Logfire Telemetry Backend**	OTLP/HTTP POST with write token over the internet
TB-2	**Application ↔ Logfire Management API**	CLI OAuth device-flow, project management calls
TB-3	**Application ↔ Logfire Query/Variables API**	Read token SQL queries; background long-poll variables
TB-4	**User Input ↔ Application**	Environment variables (`LOGFIRE_TOKEN`, `LOGFIRE_BASE_URL`, etc.), CLI arguments, pyproject.toml, credential files
TB-5	**Application ↔ OS File System**	Credential files, DiskRetryer temp queue, CLI log

ID	Boundary	Crosses
TB-6	**Application ↔ Python Import System**	Auto-trace `sys.meta_path` hook + AST rewriter
TB-7	**Application ↔ External AI/LLM APIs**	AI instrumentation captures prompt/response content
TB-8	**Package Registry ↔ Developer/CI**	PyPI distribution of `logfire`, `logfire-api`, OTel deps

---

5. Entry Points

#	Entry Point	Data Type	Trust Level	Notes
EP-01	`LOGFIRE_TOKEN` env var	Write token string (comma-sep for fan-out)	Low — caller-controlled	Embedded verbatim in OTLP `Authorization` header. Region encoded in token prefix.
EP-02	`LOGFIRE_API_KEY` env var	API key string	Low — caller-controlled	Used for managed variables API auth.
EP-03	`LOGFIRE_BASE_URL` env var	URL string	Low — caller-controlled	**No domain/allowlist validation**; can redirect all telemetry to arbitrary host.
EP-04	`LOGFIRE_SEND_TO_LOGFIRE` env var	Boolean/literal	Low — caller-controlled	Controls whether export happens at all.

#	Entry Point	Data Type	Trust Level	Notes
EP-05	`~/.logfire/default.toml`	TOML (user tokens + expiry)	Medium — local filesystem	Plaintext; used by CLI for management API calls.
EP-06	`.logfire/logfire_credentials.json`	JSON (write token + URL)	Medium — local filesystem	Per-project; read by `configure()` at startup.
EP-07	`pyproject.toml [tool.logfire]`	TOML config	Medium — local filesystem	Read-only by SDK; attacker modifying this file can change SDK behaviour.
EP-08	`logfire auth` CLI	Interactive OAuth device-flow	Low — unauthenticated start	Polls management API indefinitely; writes token to disk.
EP-09	`logfire run <script>` CLI	Script path / module	Low — developer-controlled	Executes arbitrary Python via `runpy.run_path()` with auto-instrumentation installed first.
EP-10	`logfire.configure()` Python API	All config options	Medium — application code	Central initialisation; token, base_url, scrubbing flag, processor hooks all set here.
EP-11	`logfire.span()` / `logfire.log*()`	Arbitrary span attributes	Low — application-controlled	All attribute values pass through scrubbing processor before export.
EP-12	`install_auto_tracing()` Python API	Module filter callable	Medium — application code	Installs `sys.meta_path` hook; AST-rewrites all matched modules at import time.

#	Entry Point	Data Type	Trust Level	Notes
EP-13	Pydantic plugin entry point	Pydantic validation data	Implicit — triggered by `import pydantic`	Activates before `logfire.configure()` if `logfire` is installed in the environment.
EP-14	LLM instrumentation (`instrument_openai`, etc.)	Prompt text, API responses	Low — external AI service responses	Captures full prompt/response content in span attributes by default.
EP-15	OTLP HTTP response (from backend)	HTTP status codes, headers	Low — network / backend	`raise_for_retryable_status` parses response codes; minimal parsing, not a significant attack surface.
EP-16	Disk retry queue (`/tmp/logfire-retryer-*/`)	Raw Protobuf bytes	Low — OS temp dir	Written by daemon thread; read back and POSTed to backend with stored `Authorization` header.
EP-17	`LogfireQueryClient` SQL queries	SQL string	Low — application code	SQL passed as plain string to Logfire Query API; no client-side parameterisation enforced.
EP-18	W3C `traceparent` / `tracestate` headers	Trace context strings	Untrusted — inbound network	Default behaviour (`WarnOnExtractTraceContextPropagator`) warns but accepts; `LOGFIRE_DISTRIBUTED_TRACING=true` silently accepts.

#	Entry Point	Data Type	Trust Level	Notes
EP-19	`LOGFIRE_CREDENTIALS_DIR` env var	Directory path	Low — caller-controlled	Can redirect credential storage to attacker-controlled path.

---

6. STRIDE Analysis Per Component

Component 1 — OTLP HTTP Exporter (`OTLPExporterHttpSession` / `BodySizeCheckingOTLPSpanExporter`)

Trust Boundary: TB-1 — Application ↔ Logfire Telemetry Backend

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**Yes**	Attacker impersonates the Logfire telemetry backend via DNS hijacking, BGP hijack, or by setting `LOGFIRE_BASE_URL` to a rogue server. Write token is sent to the attacker-controlled endpoint.	TLS with hostname validation (via `requests` defaults). `LOGFIRE_BASE_URL` has no domain allowlist.
T – Tampering	**Yes**	Man-in-the-middle intercepts OTLP POST, drops or modifies span payloads. No span-level signing or HMAC.	TLS in transit provides transport-layer integrity. No application-level signing.

Threat	Applicable?	Scenario	Mitigation
R – Repudiation	**Yes**	Any party with a valid write token can inject arbitrary spans into the project. SDK never signs spans, so there is no way to distinguish SDK-originated telemetry from injected spans at the backend.	No span signing is implemented. Accept-risk or add backend-side controls.
I – Info Disclosure	**Yes**	(a) Write token in `Authorization` header visible to any TLS-terminating proxy in the network path. (b) Token stored in DiskRetryer session headers (in memory) and effectively in retry task `kwargs` written to disk. (c) Span attributes may contain scrubbing-miss PII.	TLS protects in-transit token. Disk files are unencrypted and inherit OS temp dir permissions.
D – Denial of Service	**Yes**	Network unavailability causes DiskRetryer to accumulate up to 512 MB of Protobuf files in `/tmp`. On low-disk systems this exhausts storage. Daemon thread may hold the retry queue open at process exit.	Hard cap of 512 MB (`MAX_TASK_SIZE`). Daemon thread prevents process hang at exit.
E – Elevation of Privilege	**N/A**		

Component 2 — Token / Config Loader (`config_params.py`, `config.py`, `auth.py`)

Trust Boundary: TB-4 — User Input / Environment ↔ Application

Threat	Applicable?	Scenario	Mitigation

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**Yes**	`LOGFIRE_BASE_URL` env var accepted without domain allowlist or SSRF protection, allowing redirection of all OTLP export and management API calls to an arbitrary server.	Token format validated by `PYDANTIC_LOGFIRE_TOKEN_PATTERN` regex, but URL is not validated against a known-good domain list.
T – Tampering	**Yes**	Local attacker modifies `.logfire/logfire_credentials.json` or `~/.logfire/default.toml` to substitute a write token pointing to an attacker-controlled backend.	Files are TOML/JSON; no integrity signature or MAC.
R – Repudiation	**N/A**		
I – Info Disclosure	**Yes**	`LOGFIRE_TOKEN` visible in `/proc/<PID>/environ` (Linux), `ps e` output, and child processes. Tokens appear in `pyproject.toml` if users place them there (allow_file_config=False by default for TOKEN, but secrets can be set programmatically).	`TOKEN` ConfigParam has `allow_file_config=False`, preventing token in `pyproject.toml`.
D – Denial of Service	**Low**	Invalid or missing token prevents all telemetry export; raises `LogfireConfigError` at startup.	Intentional and documented.
E – Elevation of Privilege	**Yes**	`LOGFIRE_CREDENTIALS_DIR` can be pointed to a world-writable or attacker-controlled directory, causing the SDK to write credential files there. Combined with symlink attacks this could clobber arbitrary files.	Path is used directly without canonicalisation or safety checks.

Component 3 — CLI Auth Module (`cli/auth.py`, `_internal/auth.py`)

Trust Boundary: TB-2 — Application ↔ Logfire Management API, TB-5 — Application ↔ OS File System

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**Yes**	Device code flow directs user to a browser URL. An attacker who can perform DNS hijacking or who has set `LOGFIRE_BASE_URL` can redirect the auth flow to a phishing page.	TLS validation; no cert pinning. User must verify URL manually.
T – Tampering	**Yes**	`~/.logfire/default.toml` written with no file-level integrity check. A local attacker can swap tokens to redirect subsequent CLI operations.	No HMAC or signature on token file.
R – Repudiation	**Yes**	The device code flow does not bind the `device_code` to a specific client machine; any party that intercepts the device code URL could complete the auth. Activation URL is printed to the terminal.	Standard OAuth 2.0 device flow limitation.
I – Info Disclosure	**Yes**	User token written as plaintext to `~/.logfire/default.toml`. File permissions not enforced by the SDK (relies on OS umask). CLI log at `~/.logfire/log.txt` may contain HTTP request metadata.	No `chmod 600` applied to token file after write.
D – Denial of Service	**Yes**	`poll_for_token()` polls indefinitely with only a 4-consecutive-error abort. No wall-clock timeout. In automated/CI environments this could block the process indefinitely if the backend is unreachable.	Only 4 sequential errors abort. No configurable timeout.
E – Elevation of Privilege	**N/A**		

Component 4 — DiskRetryer (`exporters/otlp.py` — `DiskRetryer`)

Trust Boundary: TB-5 — Application ↔ OS File System

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**N/A**		
T – Tampering	**Yes**	Local attacker with access to `/tmp/logfire-retryer-*/` can replace retry payload files with crafted Protobuf data to inject fake spans into the Logfire project when the retry thread re-sends them. `Authorization` token is not part of the file (stored separately in session headers) but fake payloads can pollute the trace data.	Files in temp dir use UUID names but directory permissions are not set to 0700 by `mkdtemp`.
R – Repudiation	**N/A**		

Threat	Applicable?	Scenario	Mitigation
I – Info Disclosure	**Yes**	`mkdtemp()` creates a directory. On Linux, `/tmp` is often sticky-bit but world-readable until file permissions restrict access. Retry files contain raw Protobuf OTLP payloads. These are post-scrubbing, but may still contain MBI/HBI application telemetry. The session headers (containing the `Authorization: <write_token>` value) are held in the `DiskRetryer.session` object in memory — the retry task `kwargs` dict written to disk does not contain the token directly, but the URL is stored per-task in the pickled kwargs structure.	`mkdtemp()` creates directory with mode 0700 on POSIX by default. Verify this is consistent across platforms.
D – Denial of Service	**Yes**	Up to 512 MB of Protobuf files accumulate under `/tmp/logfire-retryer-*/`. On systems with low disk, this can exhaust storage. Files are not cleaned up at normal process shutdown (daemon thread exits silently).	512 MB hard cap enforced in `add_task()`. No cleanup on normal exit.
E – Elevation of Privilege	**N/A**		

Component 5 — Auto-trace Import Hook (`auto_trace/import_hook.py`, `rewrite_ast.py`)

Trust Boundary: TB-6 — Application ↔ Python Import System (privileged)

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**N/A**		
T – Tampering	**Yes**	Malformed or adversarially crafted Python source in a module that passes the `AutoTraceModule` filter could trigger unexpected behaviour in the AST rewriter. The rewriter uses `ast.parse()` and `ast.compile()`; a logic error in `rewrite_ast.py` could result in wrong code being injected into the running process.	AST parsing is done on source text, not bytecode. Python's `ast` module is robust but not adversarially hardened.
R – Repudiation	**N/A**		
I – Info Disclosure	**Yes**	Auto-tracing any module wraps every function call with `logfire.span()`, capturing function arguments, return values, and raised exceptions as span attributes. If the filter matches modules that handle cryptographic keys, authentication tokens, or user PII, this data is captured and exported. Scrubbing applies pattern-matching to attribute *names* — attribute *values* that are secrets stored under non-matching keys will leak.	`@no_auto_trace` decorator and `AutoTraceModule` filter allow exclusion. Scrubbing covers common patterns.

Threat	Applicable?	Scenario	Mitigation
D – Denial of Service	**Yes**	Overly broad filter (e.g., matching all installed packages) significantly increases span volume, memory usage, and export bandwidth. Can overwhelm the batch processor queue.	Filter is application-controlled. SDK does not enforce limits on matched modules.
E – Elevation of Privilege	**Yes**	Auto-tracing security-critical library modules (e.g., `cryptography`, `ssl`, `jwt`) can interfere with their execution context by injecting `with logfire.span():` context managers at entry/exit of security functions, potentially altering exception propagation or timing-sensitive behaviour.	No built-in protection list for security-critical modules.

Component 6 — LLM / AI Instrumentation (`integrations/llm_providers/`, `pydantic_ai.py`, `openai_agents.py`, etc.)

Trust Boundary: TB-7 — Application ↔ External AI/LLM APIs

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**N/A**		
T – Tampering	**N/A**		

Threat	Applicable?	Scenario	Mitigation
R – Repudiation	**N/A**		
I – Info Disclosure	**Yes**	LLM instrumentation captures prompt text, system messages, model responses, tool call arguments, and token counts verbatim in OTel span attributes. Users of the application may not consent to their prompts being logged. Business-sensitive prompts (IP, strategy, code) are exfiltrated to the Logfire backend.	`capture_input` / `capture_output` flags exist on some integrations. Scrubbing does not redact semantic LLM content.
D – Denial of Service	**N/A**		
E – Elevation of Privilege	**N/A**		

Component 7 — LogfireQueryClient / LogfireAPIClient (`experimental/query_client.py`, `api_client.py`)

Trust Boundary: TB-3 — Application ↔ Logfire Query/Management API

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**Yes**	Read token or API key can be exfiltrated by the same attack vectors as write tokens (env var visibility, insecure storage).	Token format validated; transmitted over HTTPS.

Threat	Applicable?	Scenario	Mitigation
T – Tampering	**N/A**		
R – Repudiation	**N/A**		
I – Info Disclosure	**Yes**	A leaked read token grants access to all stored telemetry in the project via SQL queries. Read tokens are static and long-lived (no visible rotation mechanism in the SDK).	No token rotation API exposed in the SDK. Scope limited to project-level access (enforced server-side).
D – Denial of Service	**Yes**	Unparameterised SQL strings passed directly to the query API. A caller who builds SQL from user input without sanitisation creates a server-side injection risk (impact depends on backend enforcement).	No parameterisation or input validation in `LogfireQueryClient.query()`. Enforcement is server-side only.
E – Elevation of Privilege	**N/A**		

Component 8 — Remote Variable Provider (`variables/remote.py`)

Trust Boundary: TB-3 — Application ↔ Logfire Variables API

Threat	Applicable?	Scenario	Mitigation

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**Yes**	`LOGFIRE_BASE_URL` override redirects variable fetch to attacker-controlled server; malicious variable values pushed to application.	TLS; no domain allowlist on `LOGFIRE_BASE_URL`.
T – Tampering	**Yes**	Man-in-the-middle or compromised Variables API backend can deliver tampered variable values that alter application runtime behaviour. Impact depends on how application uses variables (if in control flow, could be critical).	Values are treated as opaque strings by the SDK; Pydantic validation enforces types, but not value semantics.
R – Repudiation	**N/A**		
I – Info Disclosure	**Yes**	Background long-poll thread reveals application activity to the network. `LOGFIRE_API_KEY` visible in process environment.	Token transmitted over HTTPS.
D – Denial of Service	**Yes**	Background polling thread contends with application threads. If the variables API is unavailable, the SDK falls back to cached values (but behaviour depends on application code).	Thread is daemon; graceful fallback to cached values.
E – Elevation of Privilege	**Yes**	If application code uses remote variable values in security-sensitive contexts (auth bypass flags, feature gating security controls), a backend compromise or MITM could effectively elevate an attacker's access in the host application.	Application-level concern, but SDK design enables this pattern without warning.

Component 9 — Pydantic Plugin Entry Point (`integrations/pydantic.py`)

Trust Boundary: TB-6 — Application ↔ Python Import System (implicit activation)

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**N/A**		
T – Tampering	**N/A**		
R – Repudiation	**N/A**		
I – Info Disclosure	**Yes**	Pydantic plugin is registered as a `[project.entry-points."pydantic"]` entry point. When `pydantic` is imported, this plugin activates automatically — even before `logfire.configure()` is called. Pydantic validation data (which may contain PII, passwords, or secrets passed to models) can begin being instrumented. Users who have `logfire` installed in their environment but do not use it may be surprised by this behaviour.	`pydantic_plugin_record` defaults to `'off'` — validation data is NOT recorded unless explicitly enabled. Spans are still emitted for validation events at the metrics level.
D – Denial of Service	**N/A**		
E – Elevation of Privilege	**Yes**	Any Python application that has `logfire` installed (even as an indirect dep) will activate the Pydantic plugin globally on `pydantic` import, instrumenting all Pydantic models without explicit developer consent.	Default `pydantic_plugin_record='off'` mitigates data capture; but instrumentation framework itself activates. Principle of explicit opt-in is violated.

Component 10 — `logfire run` CLI (`cli/run.py`)

Trust Boundary: TB-4 — User Input ↔ Application / OS Execution

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**N/A**		
T – Tampering	**N/A**		
R – Repudiation	**Yes**	`logfire run <script>` executes arbitrary Python via `runpy.run_path()`. No logging of script path, arguments, or execution environment. An operator cannot audit what scripts were run with auto-instrumentation.	No audit log entry for script execution.
I – Info Disclosure	**Yes**	Auto-instrumentation is installed before the script runs. All functions in matched modules (default broad filter) are traced, including functions in the target script that may access credentials or PII.	By design; users should configure `AutoTraceModule` filter appropriately.
D – Denial of Service	**N/A**		
E – Elevation of Privilege	**Yes**	If `logfire run` is invoked with a script path provided by an untrusted source (e.g., in a CI pipeline that processes external input), it executes arbitrary code with the privileges of the calling user. No sandboxing or script validation is performed.	No isolation or validation of script path or contents.

Component 11 — Package Distribution / Supply Chain

Trust Boundary: TB-8 — Package Registry ↔ Developer/CI

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**Yes**	Typosquatting attack: a package named `logflre` or `logfire-sdk` on PyPI could trick users into installing malicious code.	PyPI package name `logfire` is claimed by Pydantic.
T – Tampering	**Yes**	Compromise of a maintainer PyPI account could allow a tampered version of `logfire` to be pushed. Narrow OTel version pin (`>=1.39.0, <1.40.0`) means security patches in OTel 1.40+ are not automatically adopted.	`uv.lock` lockfile commits exact hash-locked versions for reproducible installs. GitHub Actions publish workflow requires tag-triggered push.
R – Repudiation	**N/A**		
I – Info Disclosure	**Yes**	Any of the 30+ OTel instrumentation packages or their transitive dependencies could contain malicious code that exfiltrates telemetry, tokens, or environment variables.	`uv.lock` locks direct and transitive hashes. No automated vulnerability scanning visible in the repository's CI pipeline (Dependabot/Renovate/Trivy not detected).
D – Denial of Service	**Low**	Malicious update breaks SDK; application loses observability.	

Threat	Applicable?	Scenario	Mitigation
E – Elevation of Privilege	**Yes**	A supply-chain-compromised package runs with full application process privileges. Given the SDK's deep hooks into the Python runtime (sys.meta_path, logging bridges, OTel globals), a compromised package could access secrets in memory.	`uv.lock` hash pinning reduces risk. No SBOM published.

Component 12 — Scrubbing Processor (`_internal/scrubbing.py`)

Trust Boundary: TB-1 (data flowing across Application ↔ Backend)

Threat	Applicable?	Scenario	Mitigation
S – Spoofing	**N/A**		
T – Tampering	**N/A**		
R – Repudiation	**N/A**		

Threat	Applicable?	Scenario	Mitigation
I – Info Disclosure	**Yes**	(a) Scrubbing can be entirely disabled via `scrubbing=False` in `configure()`. (b) Default patterns only match on attribute *key names*; a secret stored under an unrecognised key (e.g., `db_pass`, `x_api_token`) bypasses scrubbing. (c) LLM prompt/response content is not covered by key-name patterns — semantic content is not scrubbed. (d) Auto-traced return values from functions handling secrets may use arbitrary variable names not in the default patterns.	16 default patterns defined in `DEFAULT_PATTERNS` Customisable via `ScrubbingOptions`.
D – Denial of Service	**N/A**		
E – Elevation of Privilege	**N/A**		

---

7. Threat Register

DREAD scoring: each factor scored 1 (low) to 3 (high). Average ≥ 2.5 → **High/Critical**; 1.5–2.4 → **Medium**; < 1.5 → **Low**.

ID	Component	STRIDE	Threat Scenario	D	R	E	A	D	Avg	Severity	Sta

ID	Component	STRIDE	Threat Scenario	D	R	E	A	D	Avg	Severity	Sta
TM-01	OTLP Exporter / Config	**S, I**	`LOGFIRE_BASE_URL` set to attacker-controlled URL redirects all OTLP telemetry (including write token in `Authorization` header) to a rogue server. Full token exfiltration and telemetry theft.	3	2	3	3	3	**2.8**	**High**	Op
TM-02	Config Loader	**I**	`LOGFIRE_TOKEN` (write token) exposed in process environment (`/proc/<PID>/environ` , `ps e` , CI logs). Leaked token allows adversary to ingest fake spans or exfiltrate token.	3	2	2	3	2	**2.4**	**Medium**	Op
TM-03	Scrubbing Processor	**I**	Scrubbing disabled (`scrubbing=False`) or secret stored under non-default attribute key name bypasses redaction; PII, passwords, or API keys exported to Logfire backend.	3	2	3	3	2	**2.6**	**High**	Op

ID	Component	STRIDE	Threat Scenario	D	R	E	A	D	Avg	Severity	Sta
TM-04	LLM/AI Instrumentation	**I**	LLM prompt text and model responses captured verbatim as span attributes and exported. Users' conversations, business-sensitive queries, and potentially PII transmitted to Logfire without user consent or awareness.	3	3	3	3	3	**3.0**	**Critical**	Op
TM-05	DB Instrumentation	**I**	SQL query parameters (including user-supplied values) captured in spans by SQLAlchemy, psycopg, asyncpg instrumentation. PII or sensitive query data exfiltrated to telemetry backend.	3	3	3	3	2	**2.8**	**High**	Op
TM-06	Auto-trace Import Hook	**I**	`install_auto_tracing()` with an overly broad module filter captures function arguments containing secrets, tokens, or user PII under attribute names not covered by the scrubbing default patterns.	3	2	2	3	2	**2.4**	**Medium**	Op

ID	Component	STRIDE	Threat Scenario	D	R	E	A	D	Avg	Severity	Sta
TM-07	DiskRetryer	**I**	Failed OTLP export payloads (raw Protobuf, post-scrubbing) written to `/tmp/logfire-retryer-*/`. If OS temp dir is world-readable or shared, other local users/processes may read application telemetry.	2	2	2	2	2	**2.0**	**Medium**	Op
TM-08	DiskRetryer	**D**	512 MB of Protobuf retry files accumulate in `/tmp` on network failure. Disk exhaustion on resource-constrained systems (containers, embedded). Files not cleaned up on graceful shutdown.	2	2	2	2	2	**2.0**	**Medium**	Op
TM-09	DiskRetryer	**T**	Local attacker replaces retry payload files with crafted Protobuf to inject fabricated spans into the Logfire project when the retry thread re-sends them. Poisons trace data for that project.	2	1	2	2	2	**1.8**	**Medium**	Op
TM-10	CLI Auth	**I**	User token written to `~/.logfire/default.toml` in plaintext with no `chmod 600` enforcement. Shared or multi-user systems expose management API tokens to other local users.	2	2	2	2	2	**2.0**	**Medium**	Op

ID	Component	STRIDE	Threat Scenario	D	R	E	A	D	Avg	Severity	Sta
TM-11	CLI Auth	**D**	Device code auth polling loop has no wall-clock timeout. In automated environments, the process blocks indefinitely if the Logfire management API is unreachable or slow.	1	2	2	1	2	**1.6**	**Medium**	Op
TM-12	OTLP Exporter	**T, R**	No span-level signing or HMAC. Any party with a valid write token can inject arbitrary spans with fake timestamps, trace IDs, and attributes. Legitimate spans cannot be distinguished from injected ones.	2	2	2	2	2	**2.0**	**Medium**	Op
TM-13	W3C Context Propagation	**T**	With `LOGFIRE_DISTRIBUTED_TRACING=true` untrusted upstream `traceparent` / `tracestate` headers are silently accepted, allowing attackers to manipulate trace IDs, parent spans, and sampling flags in the instrumented application.	2	2	2	2	2	**2.0**	**Medium**	Op

ID	Component	STRIDE	Threat Scenario	D	R	E	A	D	Avg	Severity	Sta
TM-14	Pydantic Plugin	**E, I**	Pydantic plugin auto-activates on `import pydantic` in any environment where `logfire` is installed, even without `logfire.configure()`. Developers may not be aware instrumentation is active.	2	3	3	2	2	**2.4**	**Medium**	Op
TM-15	LogfireQueryClient	**I**	Leaked read token grants permanent, broad SQL query access to all telemetry in the associated Logfire project. No read token rotation mechanism is exposed in the SDK.	3	2	2	3	2	**2.4**	**Medium**	Op
TM-16	Remote Variable Provider	**T, E**	Compromised or MITM'd Variables API backend delivers malicious variable values that alter application runtime behaviour. If variables gate security controls (auth flags, rate limits), this enables privilege escalation in the host application.	2	1	1	2	2	**1.6**	**Medium**	Op

ID	Component	STRIDE	Threat Scenario	D	R	E	A	D	Avg	Severity	Sta
TM-17	`logfire run` CLI	**E**	`logfire run <path>` executes arbitrary Python scripts via `runpy.run_path()` without sandboxing, validation, or audit logging. Untrusted script path in CI pipeline enables RCE under user's privileges.	2	2	3	1	2	**2.0**	**Medium**	Op
TM-18	Supply Chain	**T, E**	Compromise of a Pydantic maintainer's PyPI account allows a tampered `logfire` release to run arbitrary code in all user applications at install or import time. OTel version pin delays adoption of upstream security patches.	3	1	1	3	2	**2.0**	**High**	Op
TM-19	Auto-trace Import Hook	**E**	Auto-trace filter that matches security-critical libraries (`cryptography`, `ssl`, `jwt`, `hashlib`) injects `with logfire.span():` context managers into security functions, potentially altering exception propagation or timing-sensitive behaviour.	2	2	2	2	2	**2.0**	**Medium**	Op

ID	Component	STRIDE	Threat Scenario	D	R	E	A	D	Avg	Severity	Sta
TM-20	Config Loader	**E**	`LOGFIRE_CREDENTIALS_DIR` set to a world-writable path or attacker-controlled symlink causes credential files (with write tokens) to be written to an attacker-readable location.	2	1	2	1	2	**1.6**	**Medium**	Op

---

8. Mitigations Summary

8.1 Critical / High Priority

M-01 — Add `LOGFIRE_BASE_URL` allowlist / SSRF protection (addresses TM-01)

- **Threat**: TM-01 — Base URL override redirects telemetry to attacker-controlled server
- **Type**: Prevent
- **Control**: Validate `LOGFIRE_BASE_URL` against an allowlist of known Logfire regions (`logfire-us.pydantic.dev` , `logfire-eu.pydantic.dev`) or enforce HTTPS-only with a documented override mechanism requiring an explicit `unsafe_allow_custom_url=True` flag. At minimum, emit a prominent warning when the default domain is overridden.
- **Files**: `logfire/_internal/config_params.py` (TOKEN, BASE_URL params), `logfire/_internal/auth.py` (REGIONS dict)
- **Owner**: SDK maintainers
- **Status**: **Open**

M-02 — Gate LLM content capture behind explicit opt-in (addresses TM-04)

- **Threat**: TM-04 — LLM prompts/responses captured and exported without user awareness
- **Type**: Prevent
- **Control**: Default `capture_input=False` and `capture_output=False` for all LLM/AI instrumentation wrappers (OpenAI, Anthropic, Google Gemini, LiteLLM, PydanticAI, MCP). Document that enabling capture constitutes logging of potentially sensitive user data. Add a privacy notice in documentation and emit a `UserWarning` when content capture is enabled.
- **Files**: `logfire/_internal/integrations/llm_providers/llm_provider.py`, all `instrument_*` wrappers in `llm_providers/`
- **Owner**: SDK maintainers
- **Status**: **Open** (some integrations already have flags; audit for consistent defaults)

M-03 — Scope and document scrubbing gaps; add semantic content scrubbing (addresses TM-03, TM-05)

- **Threat**: TM-03, TM-05 — Scrubbing bypass via non-standard attribute names or SQL parameter values
- **Type**: Prevent / Detect
- **Control**: (a) Extend `DEFAULT_PATTERNS` to cover common variations (`db_pass`, `x_api_token`, `bearer`, `access_token`, `refresh_token`, `client_secret`). (b) Add a scrubbing mode for SQL span attribute values (not just key names) that redacts values matching DB parameter patterns. (c) Document clearly that `scrubbing=False` is a security risk and emit a `SecurityWarning` when it is set. (d) Add scrubbing coverage for `db.statement` attribute values.
- **Files**: `logfire/_internal/scrubbing.py`, `logfire/_internal/config.py`
- **Owner**: SDK maintainers
- **Status**: **Open**

M-04 — Enforce strict file permissions on credential files (addresses TM-10, TM-07)

- **Threat**: TM-10 — User tokens in plaintext world-readable files
- **Type**: Prevent
- **Control**: After writing `~/.logfire/default.toml` and `.logfire/logfire_credentials.json`, call `os.chmod(path, 0o600)` (POSIX) to restrict read/write to the owner only. On Windows, use `icacls` or the `win32security` API to restrict ACLs. Verify the effective permissions match the requirement on each read.
- **Files**: `logfire/_internal/auth.py` (token file write), `logfire/_internal/config.py` (credentials file write)
- **Owner**: SDK maintainers
- **Status**: **Open**

M-05 — Add automated dependency vulnerability scanning to CI (addresses TM-18)

- **Threat**: TM-18 — Supply chain compromise via unpatched dependencies
- **Type**: Detect
- **Control**: Enable GitHub Dependabot security alerts and `dependabot.yml` for the Python ecosystem. Add a `pip audit` or `uv audit` step to `.github/workflows/main.yml` that fails the build on known CVEs in direct and transitive dependencies. Publish a Software Bill of Materials (SBOM) with each release using `cyclonedx-python` or `syft`.
- **Files**: `.github/workflows/main.yml`, add `.github/dependabot.yml`
- **Owner**: SDK maintainers / DevSecOps
- **Status**: **Open**

8.2 Medium Priority

M-06 — Add wall-clock timeout to device code polling loop (addresses TM-11)

- **Threat**: TM-11 — Indefinite auth polling blocks automated processes
- **Type**: Prevent
- **Control**: Add a `timeout_seconds` parameter to `poll_for_token()` (defaulting to 300 seconds / 5 minutes). In CI detection contexts (`PYTEST_VERSION`, `CI` env vars), default to a shorter timeout or fail fast.
- **Files**: `logfire/_internal/cli/auth.py` (`poll_for_token`)
- **Owner**: SDK maintainers
- **Status**: **Open**

M-07 — Clean up DiskRetryer temp directory on shutdown (addresses TM-07, TM-08)

- **Threat**: TM-07 (temp file exposure), TM-08 (disk exhaustion)
- **Type**: Prevent / Respond
- **Control**: Register an `atexit` handler in `DiskRetryer` to delete the `logfire-retryer-*` temp directory and its contents on normal process exit. For abnormal exits, document the retry directory location so operators can add it to log rotation or tmpfiles.d cleanup. Additionally verify that `mkdtemp()` creates the directory with mode 0700 on all supported platforms.
- **Files**: `logfire/_internal/exporters/otlp.py` (`DiskRetryer.__init__`)
- **Owner**: SDK maintainers
- **Status**: **Open**

M-08 — Validate `LOGFIRE_CREDENTIALS_DIR` path against traversal and symlink attacks (addresses TM-20)

- **Threat**: TM-20 — Credentials dir override leads to token file written to attacker-accessible path
- **Type**: Prevent

- **Control**: Resolve `LOGFIRE_CREDENTIALS_DIR` to a canonical absolute path using `Path.resolve()` before use. Warn if the directory is world-writable. Do not follow symlinks when writing credential files (use `O_NOFOLLOW` on POSIX).
- **Files**: `logfire/_internal/config_params.py`, `logfire/_internal/config.py`
- **Owner**: SDK maintainers
- **Status**: **Open**

M-09 — Document `LOGFIRE_DISTRIBUTED_TRACING` security implications (addresses TM-13)

- **Threat**: TM-13 — Untrusted traceparent accepted
- **Type**: Detect
- **Control**: Add a documentation warning to `LOGFIRE_DISTRIBUTED_TRACING` explaining the trace injection risk. In the default `WarnOnExtractTraceContextPropagator`, include the remote `traceparent` value in the warning so operators can identify injection attempts. Consider sampling-flag validation to reject `sampled=1` from untrusted contexts unless explicitly configured.
- **Files**: `logfire/_internal/config.py`, `logfire/propagate.py`
- **Owner**: SDK maintainers
- **Status**: **Open**

M-10 — Add explicit opt-in mechanism for Pydantic plugin (addresses TM-14)

- **Threat**: TM-14 — Pydantic plugin activates without explicit developer consent
- **Type**: Prevent
- **Control**: Check for the presence of a `LOGFIRE_PYDANTIC_PLUGIN_ENABLED=true` environment variable or `pydantic_plugin=True` in `pyproject.toml [tool.logfire]` before activating the plugin. Without this opt-in, the plugin entry point should register as a no-op stub. Document this as a breaking change from the current implicit-activation model.
- **Files**: `logfire/integrations/pydantic.py`, `logfire/_internal/config.py`

- **Owner**: SDK maintainers
- **Status**: **Open**

M-11 — Warn when auto-trace filter matches security-critical modules (addresses TM-19)

- **Threat**: TM-19 — Auto-trace instruments security-critical library code
- **Type**: Detect
- **Control**: Maintain a built-in blocklist of known security-sensitive module prefixes (`cryptography` , `ssl` , `jwt` , `hashlib` , `hmac` , `secrets` , `keyring` , `pyotp`). Emit a `SecurityWarning` if the `AutoTraceModule` filter matches any blocked prefix. Allow override with an explicit `allow_security_modules=True` flag.
- **Files**: `logfire/_internal/auto_trace/__init__.py` , `logfire/_internal/auto_trace/import_hook.py`
- **Owner**: SDK maintainers
- **Status**: **Open**

M-12 — Add audit logging for `logfire run` script execution (addresses TM-17)

- **Threat**: TM-17 — Arbitrary script execution without audit trail
- **Type**: Detect
- **Control**: Log the resolved script path, working directory, and Python version to the CLI activity log (`~/.logfire/log.txt`) before executing the script via `runpy.run_path()` . Print a summary to stdout including the canonical script path, so operators can verify what was run.
- **Files**: `logfire/_internal/cli/run.py`
- **Owner**: SDK maintainers
- **Status**: **Open**

M-13 — Document read token scope and rotation procedure (addresses TM-15)

- **Threat**: TM-15 — Read token leakage exposes all project telemetry
- **Type**: Detect / Respond
- **Control**: Document that read tokens provide broad project-level query access. Add a `logfire read-tokens revoke` CLI command. In `LogfireQueryClient` docstrings, note the sensitivity of the read token and recommend storing it in a secrets manager rather than environment variables or source code.
- **Files**: `logfire/_internal/cli/__init__.py`, `logfire/experimental/query_client.py`
- **Owner**: SDK maintainers
- **Status**: **Open**

8.3 Low Priority / Defense-in-Depth

M-14 — Add HMAC / integrity check for DiskRetryer retry files (addresses TM-09)

- **Threat**: TM-09 — Retry file replacement injects fake spans
- **Type**: Prevent
- **Control**: Before writing each retry payload, compute a keyed HMAC (using the write token as the key) and store it alongside the payload. Verify the HMAC before re-sending. This prevents tampering by local users who cannot compute the HMAC without the write token.
- **Files**: `logfire/_internal/exporters/otlp.py` (`DiskRetryer`)
- **Owner**: SDK maintainers
- **Status**: **Open**

M-15 — Extend OTel version pin policy to include security patch adoption (addresses TM-18)

- **Threat**: TM-18 — Delayed OTel security patches
- **Type**: Prevent
- **Control**: Establish a policy to update the OTel version constraint within 30 days of a security advisory in `opentelemetry-sdk` or `opentelemetry-exporter-otlp-proto-http`. Track OTel SDK security advisories via GitHub Advisory Database subscriptions or `dependabot.yml`.
- **Files**: `pyproject.toml`, `.github/dependabot.yml`
- **Owner**: SDK maintainers
- **Status**: **Open**

---

9. Out of Scope

The following areas were considered but are explicitly **out of scope** for this threat model:

Area	Reason
Logfire SaaS backend (server, dashboard, storage)	Closed-source; not present in this repository. Server-side auth enforcement, data access controls, and storage security must be assessed separately by the Logfire platform team.
Logfire web dashboard XSS / CSRF	Closed-source frontend; not in this repository.
Host application security	The SDK operates within the host Python application's process. Vulnerabilities in the host application (e.g., injection, SSRF, RCE) are out of scope.

Area	Reason
Pyodide / browser deployment	Experimental deployment target with different security characteristics. Threading and disk I/O are disabled in Emscripten. Privacy implications of browser-based telemetry require a separate privacy impact assessment.
OpenTelemetry SDK vulnerabilities	CVEs in upstream `opentelemetry-sdk` or OTLP exporter packages are tracked by the OTel project; the Logfire SDK inherits fixes by updating its OTel version constraint (see M-15).
Closed-source Logfire platform RBAC	Token-scoped authorisation is enforced server-side; the SDK has no RBAC logic to audit.
JavaScript / Pyodide test harness	`pyodide_test/` is a test harness only; not distributed to end users.
`logfire-api` shim package	Zero-dependency no-op stubs; no data flows, no credentials, no network calls. No meaningful attack surface.

Summary

Metric	Value
Total threats identified	20
Critical	1 (TM-04)
High	4 (TM-01, TM-03, TM-05, TM-18)
Medium	15 (TM-02, TM-06 through TM-17, TM-19, TM-20)

Metric	Value
Low	0
Open mitigations	15
Accepted risk	0

Top 3 Open Mitigations (by impact)

1. **M-02 — Gate LLM content capture behind explicit opt-in** *(Critical — TM-04)* LLM instrumentation currently captures full prompt and response content by default. In any AI-enabled application this means user conversations, business-sensitive queries, and PII are streamed to the Logfire backend without user consent or notice. Defaulting to `capture_input=False` / `capture_output=False` is the most impactful single change to reduce privacy risk.

2. **M-01 — Add `LOGFIRE_BASE_URL` domain allowlist / SSRF protection** *(High — TM-01)* Any process with permission to set environment variables can redirect all OTLP telemetry (including the write token in the `Authorization` header) to an arbitrary server. This is a complete credential exfiltration and telemetry hijack vector. Validating the override URL against known Logfire domains eliminates the attack path entirely.

3. **M-03 — Extend scrubbing patterns and add SQL parameter value scrubbing** *(High — TM-03, TM-05)* The scrubbing processor only matches on attribute key names; secrets stored under non-default keys, SQL parameter values, and LLM response content all bypass redaction. Extending the default pattern list and adding value-level scrubbing for known sensitive attribute types (e.g., `db.statement`) significantly reduces the risk of PII/credential exfiltration to the telemetry backend.